

Topic 6: Functions Returning Pointers

1. High-Yield Conceptual Theory

1.1 Definition and Declaration Syntax

A function can return a pointer to any data type (`int *`, `char *`, `struct Node *`, `void *`, or even a pointer to another pointer `int **`).

Syntax

```
return_type *function_name(parameter_list);
```

Precedence Revisit

- `int *f(void);` → `()` has higher precedence than `*`. This is a **function** taking `void` and returning a **pointer to int** (`int *`).
- `int (*f)(void);` → Parentheses force `*` to bind to `f` first. This is a **pointer to a function** returning `int` .

Call Stack Execution Lifecycle:

Caller Frame

[...]

Callee Frame
(Active)

[Local Var 'x']
[Return Addr] | Address &x

Function Returns → Callee Frame Popped (Deallocated!)
Returned Pointer → Points to Dead Stack Memory! (Dangling Pointer)

1.2 The Core Dilemma: Memory Lifetime vs. Address Validity

When returning a pointer, the critical question is always: **Does the pointed-to memory remain valid after the function's activation record is popped off the stack?**

Depending on where the target variable resides in memory, returning its address produces either safe code or **Undefined Behavior (UB)**:

Memory Segment	Lifetime	Safe to Return?	Example	Risk / Side Effect
Stack (Automatic Variables)	Block/Function scope	NO (UB)	<code>int x; return &x;</code>	Dangling pointer ; stack frame will be overwritten.
Heap (Dynamic Allocation)	Until <code>free()</code> is called	YES	<code>int *p = malloc(...); return p;</code>	Safe, but caller must free memory to prevent memory leaks .
Data / BSS (Static Variables)	Entire program execution	YES	<code>static int x; return &x;</code>	Safe, but not re-entrant ; subsequent calls overwrite value.
Read-Only Data (String Literals)	Entire program execution	YES	<code>return "GATE CSE";</code>	Safe to read; writing via pointer triggers Segmentation Fault .
Caller's Stack Frame	Valid throughout call chain	YES	<code>char *p; return p + 2;</code>	Safe (caller's frame outlives callee's frame).

1.3 Deep-Dive: The Local Array vs. String Literal Trap

This is one of the most frequently tested distinctions in GATE:

Case A: Returning a Local Array (Illegal)

```
char *getString() {  
    char str[] = "GATE"; // Array allocated on CALLEE'S STACK FRAME  
    return str;           // Returns address of stack memory -> DANGLING POINTER!  
}
```

- `str` is a local character array. The string characters `'G'`, `'A'`, `'T'`, `'E'`, `'\0'` are copied into the **stack frame** of `getString()` .
- When `getString()` returns, its stack frame is invalidated. Any subsequent function call (like `printf`) overwrites that memory. Dereferencing the returned pointer is **Undefined Behavior**.

Case B: Returning a Pointer to a String Literal (Legal)

```
char *getString() {  
    char *str = "GATE"; // Pointer to string literal in READ-ONLY DATA SEGMENT  
    return str;         // Perfectly SAFE!  
}  
  
// Equivalent to:  
char *getStringDirect() {  
    return "GATE";      // Perfectly SAFE!  
}
```

- `"GATE"` is stored in the constant string table (Read-Only Data / Text Segment), which exists for the **entire duration** of the program.
- Returning its address is 100% safe. However, attempting to modify it (`str[0] = 'M';`) produces a **Segmentation Fault / Undefined Behavior**.

1.4 Returning Static Variables & The Reentrancy Problem

When a function returns a pointer to a `static` variable, the data persists across function calls:

```
int *compute(int val) {
    static int result;
    result = val * 2;
    return &result;
}
```

The Shared-Buffer Caveat

Because only **one** instance of a static variable exists:

```
int *p1 = compute(5); // *p1 = 10
int *p2 = compute(20); // *p2 = 40

printf("%d %d\n", *p1, *p2); // Prints: 40 40 (NOT 10 40!)
```

- Both `p1` and `p2` point to the exact same memory address (`&result`). The second call overwrote the value.
- This pattern is common in legacy C library functions (e.g., `strtok()`, `ctime()`), making them **non-thread-safe / non-reentrant**.

1.5 Chaining and Dereferencing Returned Pointers

Because a function call evaluates to a pointer, it can be directly dereferenced or indexed:

```
int *get_array();
struct Node *get_head();

int val = *get_array();           // Dereferences element at index 0
int third = get_array();          // Pointer subscripting directly on return value
int data = get_head()->data;      // Member access via structure pointer
```

2. Common GATE Pitfalls & Edge Cases

1. Returning `malloc` Memory Without NULL Check

If `malloc` fails (out of heap memory), it returns `NULL`. If the caller immediately dereferences the returned pointer without checking for `NULL`, a crash occurs:

```
int *create() {
    return (int *)malloc(sizeof(int));
}

int main() {
    int *p = create();
    *p = 10; // Crash if heap allocation failed (Dereferencing NULL)
}
```

2. Double-Free Bugs Across Function Boundaries

If a function returns a dynamically allocated pointer, ownership passes to the caller. A double-free occurs if both caller and an internal routine try to free it, or if it is freed twice:

```
free(ptr);
free(ptr); // UNDEFINED BEHAVIOR: Double free corruption
```

3. Modifying Read-Only String Literals Returned from Functions

```
char *greet() {
    return "Hello";
}

int main() {
    char *s = greet();
    s[0] = 'h'; // RUNTIME ERROR: Bus Error / Segmentation Fault (Writing to RO Data)
}
```

3. Previous Year GATE Questions with Step-by-Step Solutions

PYQ 1 (GATE CS 2012)

Question:

Consider the following C function:

```
int *f() {
    int x = 5;
    return &x;
}
```

- Which of the following statements is true regarding function `f()` ?
- (A) It produces a compiler error because addresses of variables cannot be returned.
 - (B) It causes a run-time error when called.
 - (C) It returns the address of a local variable, which leads to undefined behavior when dereferenced by the caller.
 - (D) It safely returns the value 5 stored at pointer `&x`.

Step-by-step Solution:

- In C, returning an address of any variable is syntactically valid; the compiler will not reject it as a fatal syntax error (though modern compilers emit a warning). Thus, (A) is false.
- The function call itself returns normally without an immediate hardware trap or runtime abort. Thus, (B) is false.
- Variable `x` has automatic storage duration (allocated on the function's stack frame).
- When `f()` finishes executing, its activation record is deallocated.
- The returned address points to memory that is no longer reserved for `x`.
- Dereferencing this pointer in the caller accesses invalid/reused stack memory, which constitutes **Undefined Behavior**.

Correct Option: (C)

PYQ 2 (GATE CS 2019 — Memory Model & Pointer Mechanics)

Question:
Consider the following C program:

```
#include <stdio.h>

int *m() {
    static int a = 10;
    return &a;
}

int main() {
    int *p = m();
    printf("%d ", *p);
    *p = 20;
    printf("%d", *m());
    return 0;
}
```

What is the output printed by the program?

Step-by-step Solution:

- Inside `m()`, variable `a` is declared `static`.
 - It is allocated in the **Data Segment**, not on the stack.
 - Its lifetime is the entire duration of the program.
- First call: `int *p = m();`
 - `p` holds the address of `a`.
 - `printf("%d ", *p);` dereferences `p` and prints `10`.
- Mutation: `*p = 20;`
 - The value stored at address `&a` in the data segment is updated from `10` to `20`.
- Second call: `*m()`
 - `m()` returns `&a`.
 - `*m()` immediately dereferences `&a`, reading the updated value `20`.
 - `printf("%d", *m());` prints `20`.
- Combined output: `10 20`.

Output: 10 20

PYQ 3 (GATE CS 2008)

Question:
Consider the following C code segment:

```
char *c[] = { "GATE2024", "GATE2025", "GATE2026", "GATE2027" };
char **cp[] = { c + 3, c + 2, c + 1, c };
char ***cpp = cp;
```

```
int main() {
    printf("%s", **++cpp);
    return 0;
}
```

What is printed by the program?

Step-by-step Solution:

- Memory initialization:
 - c[0] = "GATE2024", c = "GATE2025", c = "GATE2026", c[3] = "GATE2027".
 - cp[0] = c + 3 (points to c[3]).
 - cp = c + 2 (points to c).
 - cp = c + 1 (points to c).
 - cp[3] = c (points to c[0]).
 - cpp = cp (points to cp[0]).
- Evaluate expression: **++cpp
 - Pre-increment ++cpp :
 - Moves pointer cpp from cp[0] to cp.
 - First dereference *cpp :
 - Fetches value at cp, which is c + 2 (points to c).
 - Second dereference **cpp :
 - Fetches value at c, which is "GATE2026".
- printf("%s", ...) prints the null-terminated string at c.

Output: GATE2026

4. Tricky Practice Questions (Edge Cases & Traps)

Practice Question 1: Stack Frame Re-use Overwriting Dangling Pointers

Question:

What is the expected behavior and output of the following C program?

```
#include <stdio.h>

int *f1() {
    int val = 100;
    return &val;
}

void f2() {
    int dummy = 888;
}

int main() {
    int *p = f1();
    f2();
    printf("%d\n", *p);
    return 0;
}
```

Detailed Explanation:

- f1() allocates val = 100 in its stack frame and returns &val.
- When f1() exits, its stack frame is popped. p now points to unreserved stack memory (dangling pointer).
- f2() is called immediately afterward:
 - Its activation record is pushed onto the stack, occupying the **exact same memory location** previously used by f1().
 - The local variable dummy = 888 is written into that memory space.
- When f2() returns, printf("%d\n", *p) dereferences p.
- Instead of reading 100, it reads 888 (or garbage, depending on compiler register optimizations and stack layout).
- This demonstrates why returning stack pointers causes unpredictable bugs via **Undefined Behavior**.

Result: Undefined Behavior (Practically outputs 888 or garbage)

Practice Question 2: Successive Calls Overwriting Static Buffer

Question:

Consider the following program:

```
#include <stdio.h>

char *to_binary(int n) {
```

```
static char buf[5];
for (int i = 3; i >= 0; i--) {
    buf[i] = (n & 1) ? '1' : '0';
    n >>= 1;
}
buf[4] = '\0';
return buf;
}

int main() {
    char *s1 = to_binary(3);
    char *s2 = to_binary(5);
    printf("%s %s\n", s1, s2);
    return 0;
}
```

What is the exact output printed by the program?

Detailed Explanation:

1. `buf` is a static array inside `to_binary()`. There is only **one** instance of `buf` shared across all invocations.
2. First call: `to_binary(3)`:
 - 3 in 4-bit binary is `0011`.
 - `buf` becomes `"0011"`.
 - Returns `&buf[0]`. Pointer `s1` points to `buf`.
3. Second call: `to_binary(5)`:
 - 5 in 4-bit binary is `0101`.
 - The **same buffer** `buf` is overwritten with `"0101"`.
 - Returns `&buf[0]`. Pointer `s2` points to `buf`.
4. Now, both `s1` and `s2` point to the exact same buffer in static memory containing `"0101"`.
5. `printf("%s %s\n", s1, s2);` prints the contents of `buf` twice.

Output: 0101 0101

Practice Question 3: Dynamic Pointer Allocation inside Function

Question:

What will be the output of the following C code?

```
#include <stdio.h>
#include <stdlib.h>

int *allocate(int size) {
    int *arr = (int *)malloc(size * sizeof(int));
    for (int i = 0; i < size; i++) {
        arr[i] = (i + 1) * 10;
    }
    return arr;
}

int main() {
    int *p = allocate(3);
    printf("%d %d\n", *p, *(p + 2));
    free(p);
    return 0;
}
```

Detailed Explanation:

1. `allocate(3)` requests heap memory for 3 integers: `arr[0]`, `arr`, `arr`.
2. Heap memory persists until explicitly freed.
3. The loop populates:
 - `arr[0] = 10`
 - `arr = 20`
 - `arr = 30`
4. The pointer `arr` is returned to `main()` and stored in `p`.
5. `*p` evaluates to `p[0] = 10`.
6. `*(p + 2)` evaluates to `p = 30`.
7. `free(p)` deallocates the heap block safely.
8. Output: `10 30`.

Output: 10 30

5. Quick Revision & Edge-Case Checklist

- ☐ **Never** return the address of an automatic local variable (`int x; return &x;` is always Undefined Behavior).
 - ☐ Returning a string literal (`return "ABC";`) is safe because it is stored in the read-only data segment, but the returned string **must not be modified**.
 - ☐ Returning a local array (`char s[] = "ABC"; return s;`) returns a pointer to a deallocated stack frame (dangling pointer).
 - ☐ Pointers to heap-allocated memory (`malloc / calloc`) are completely safe to return, but require the caller to call `free()` .
 - ☐ Functions returning pointers to `static` variables are **not reentrant**; subsequent calls overwrite the shared buffer.
 - ☐ A function call returning a pointer can be indexed directly: `get_arr()[i] == *(get_arr() + i)` .
-

This concludes **Topic 6: Functions Returning Pointers**. Reply with **"Continue"** to proceed to the next topic: **Arrays (1D, 2D, Multidimensional, Pointer Arithmetic & Memory Mapping)**.